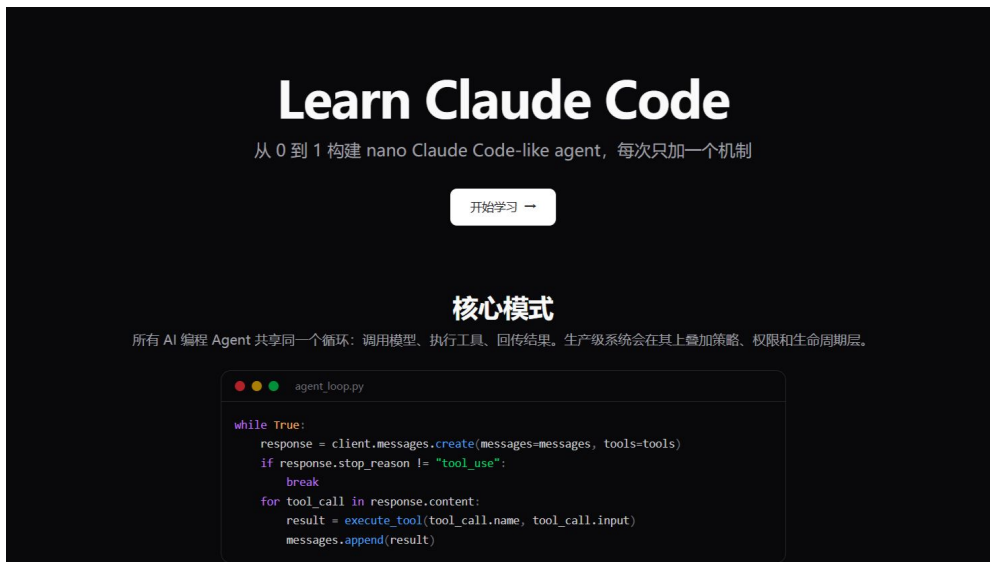


课程笔记

Learn Claude Code 源码详解 - 完结： Harness Engineering

五道口纳什 & Codex

2026 年 4 月 13 日



视频作者/频道：五道口纳什实战训练营

发布日期：2025-04-07

视频时长：6:39

视频链接：<https://www.bilibili.com/video/BV1gRDeB3ETN/>

目录

1 Harness Engineering: 马具隐喻与核心思想	2
1.1 从源码解析到 Harness 工程	2
1.2 马具隐喻: LLM 是一匹需要约束的马	2
1.3 Harness 的边界与组成	3
1.4 本章小结	3
2 Web 端架构: 五大正交关注点与 12 章内容全景	3
2.1 五大正交关注点: 完整 Agent 的骨架	3
2.2 Harness 的五大外部组件	4
2.3 安全基石: Permission、沙箱隔离与操作审批	5
2.4 12 章内容全景回顾	5
2.5 本章小结	7
3 核心机制: Agent Loop、工具管理与上下文权限	7
3.1 Agent Loop: 几乎所有 Agent 都逃不过的模式	7
3.2 Harness 的三大核心职责	8
3.3 统一目标: 让 LLM 始终往正确的方向走	9
3.4 本章小结	10
4 学习展望: 从 Claude Code 到更复杂的 Harness 工程	10
4.1 模型与 Harness 的解耦思想	10
4.2 Claude Code 的入门价值	11
4.3 向更复杂系统进阶	11
4.4 本章小结	12
5 Skills.sh 实践演示与系列结语	12
5.1 Skills.sh: Agent 的能力市场	12
5.2 Front End Slides Skill 实战演示	13
5.3 系列结语	15
5.4 本章小结	15
6 总结与延伸	15
6.1 视频核心回顾	15
6.2 从认知到实践的延伸思考	15
6.3 下一步学习建议	16

1 Harness Engineering：马具隐喻与核心思想

1.1 从源码解析到 Harness 工程

本视频作为《Learn Claude Code 源码详解》系列的完结篇，并未引入更多新的源码细节，而是将视角上升到架构层，对 Claude Code 的 Web 端实现进行了一次系统性的总结。主讲人开篇即指出，Claude Code 本质上不是一个简单的”大模型套壳”项目，而是一个典型的 **Harness Engineering** (马具工程) 项目。这一工程定位，决定了我们在阅读源码时，应当关注的不仅是模型本身如何生成文本，更重要的是：系统如何通过一系列外部机制，让模型的输出始终可控、可用、符合预期。

什么是 Harness Engineering ?

Harness 一词原意为”套索、马具”。在 AI 工程语境中，Harness Engineering 指的是围绕大语言模型 (LLM) 构建的一整套外部控制框架，包括工具调用、权限管理、上下文控制、知识感知、动作约束等。它的目标不是替换 LLM，而是像马具套住马匹一样，引导和约束模型的行为，使其朝着正确的方向前进。

1.2 马具隐喻：LLM 是一匹需要约束的马

视频中最具启发性的讲解，是将 LLM 比作一匹可能”乱跑”的马。这匹马拥有强大的奔跑能力 (即生成与推理能力)，但它并不总是清楚用户真正想要的方向。正如主讲人所强调的：

大语言模型就像一匹马，它可能乱跑。每次给出的结果——response 里面的 `tool_use` block——它调用的工具可能并不是你想让它调用的。

在这种情况下，我们需要”马具”来限制这匹马，让它往正确的方向走，给出正确的结果。这匹”马”就是 LLM 本身，而”马具”则是除模型之外的所有外部组件。

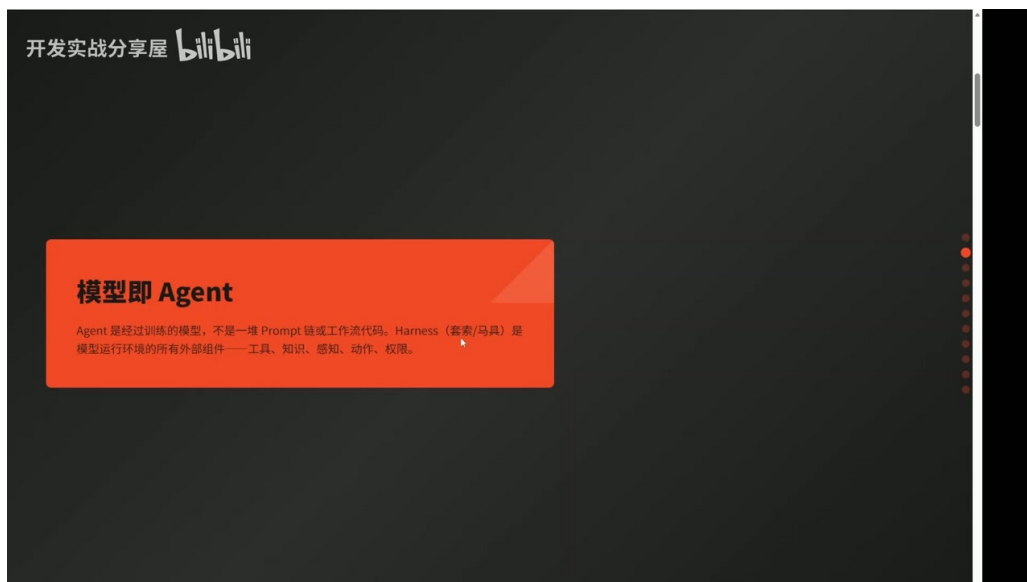


图 1：核心定义：模型即 Agent，Harness 是模型运行环境的所有外部组件

1

¹ 视频画面时间区间：00:01:05–00:01:20。

1.3 Harness 的边界与组成

那么, Harness 具体包含什么? 根据视频中的 PPT 展示, Harness 是大模型之外的整个世界, 它至少涵盖以下五大组件:

- **工具 (Tools)**: 模型可调用的外部 API、函数、代码执行环境等;
- **知识 (Knowledge)**: 注入模型的领域知识、记忆、RAG 上下文等;
- **感知 (Observation)**: 模型对当前环境状态的观察与反馈;
- **动作 (Action)**: 模型决定执行的具体操作及其后果;
- **权限 (Permissions)**: 对模型能做什么、不能做什么的安全边界约束。

Agent

视频反复强调的一个核心等式是:

$$\text{Agent} = \text{Harness} + \text{LLM}(\text{Model})$$

Agent 并非单纯的 Prompt 链或工作流代码, 而是大模型与外部控制框架的有机结合。Claude Code 项目中的所有”巧思”——从权限沙箱到 Agent Loop——本质上都是 Harness 的组成部分。

这些外部组件共同构成了 Claude Code 的 Harness, 也是本系列源码解析的真正核心所在。理解了 Harness 的边界, 我们就能够以一个更清晰的框架去审视后续章节中涉及的每一个模块。

1.4 本章小结

本章通过”马具”这一生动隐喻, 建立了理解 Claude Code 源码的顶层框架。我们明确了以下几点: 第一, Claude Code 是一个 Harness Engineering 项目, 其价值在于围绕 LLM 构建了一套精密的外部控制机制; 第二, LLM 本身像一匹能力强大但可能偏航的马, Harness 的作用就是约束和引导它; 第三, Harness 的范围涵盖工具、知识、感知、动作与权限五大组件, 而 Agent 正是 LLM 与 Harness 的合体。这一认知框架, 将贯穿后续所有章节的讨论。

2 Web 端架构: 五大正交关注点与 12 章内容全景

2.1 五大正交关注点: 完整 Agent 的骨架

进入 Web 端架构层后, 主讲人首先展示了 Claude Code 的整体架构划分。这一架构并非单一模块的堆砌, 而是由五个相互独立、彼此正交的关注点 (Orthogonal Concerns) 组合而成。它们共同搭起了完整 Agent 系统的骨架。

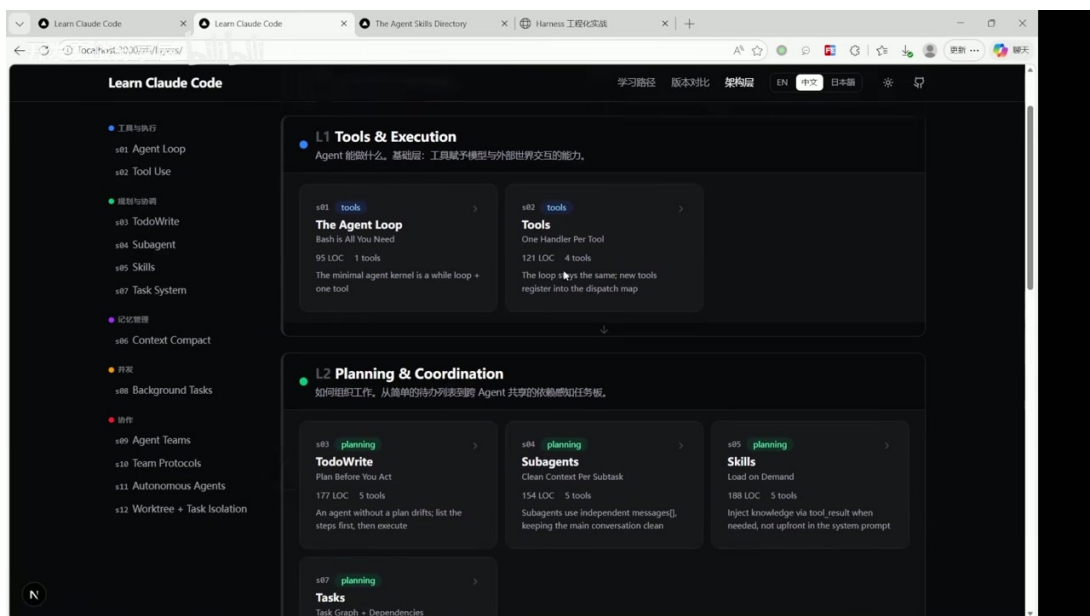


图 2: 网页端架构层的五个正交关注点，组合成完整的 Agent 系统

2

从架构设计的角度看，”正交”意味着这五个关注点在逻辑上相互独立：修改其中一个模块的实现，不应意外破坏其他模块的行为。这种设计保证了系统的可维护性与可扩展性，也是高质量 Harness 工程的标志。

2.2 Harness 的五大外部组件

在更具体的 PPT 展示中，主讲人进一步将 Harness 拆解为模型外部的五大组件：工具、知识、感知、动作与权限。这五大组件构成了 Agent 与真实世界交互的完整界面。

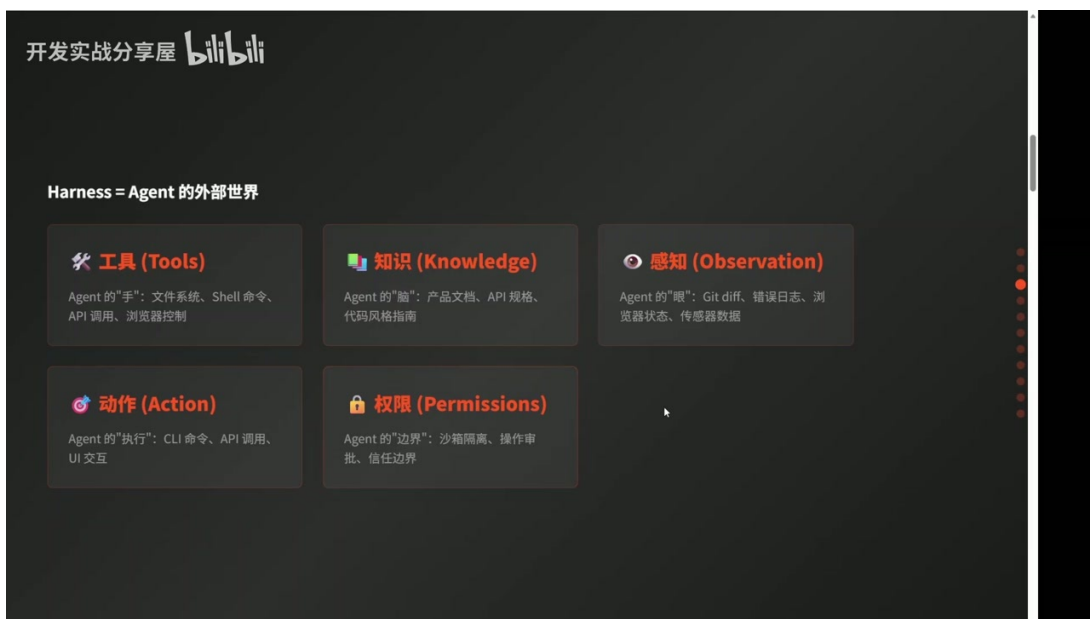


图 3: Harness = Agent 的外部世界，由工具、知识、感知、动作、权限五大组件构成

3

²视频画面时间区间：00:00:05–00:00:13。

Harness 的五大组件

1. **Tools (工具)**: 给 Agent 的”双手”，让它能执行代码、调用 API、读写文件；
2. **Knowledge (知识)**: 给 Agent 的”领域 expertise”，让它能基于上下文做出专业决策；
3. **Observation (感知)**: 让 Agent 看到环境反馈，形成闭环；
4. **Action (动作)**: Agent 对外部世界施加影响的能力；
5. **Permissions (权限)**: 为 Agent 划定边界，防止越权操作。

2.3 安全基石：Permission、沙箱隔离与操作审批

在所有 Harness 组件中，安全机制是最不能被忽视的基础层。视频特别强调了 Claude Code 中的两项安全设计：沙箱隔离（Sandbox）与操作审批（Approval）。

沙箱隔离的核心目标是防止 Agent 执行危险操作。无论是删除系统文件、访问敏感数据，还是执行未经验证的代码，沙箱机制都会对其进行隔离和限制。这就好比给马匹划定了一个安全的跑道，即使它受惊狂奔，也不会冲出围栏造成破坏。

操作审批则是用户在日常使用 Claude Code 时最常接触到的机制。当子代理（Subagent）需要执行某些敏感操作时，它会先向主代理发送一个请求，随后系统会弹出一个窗口，由用户选择”同意（Approve）”或”拒绝（Deny）”。主讲人指出，这种审批流程是 Harness 中权限控制的具体体现，也是保证人机协作安全性的关键一环。

2.4 12 章内容全景回顾

视频进一步以一张 PPT 总览图，回顾了源码解析系列前 12 个章节的内容。这些章节按照功能演进被划分为四个阶段：

³视频画面时间区间：00:01:58–00:02:10。



图 4: 12 个渐进章节总览：从核心循环到多代理团队

4

- **第一阶段 (01–02) 核心循环**：代理循环 (Agent Loop) 与工具调用 (Tool Calling)，是整个系统最底层的运行模式；
- **第二阶段 (03–06) 规划与知识**：代办事项 (To-do List)、子代理 (Subagent)、技能加载 (Skills)、上下文压缩 (Context Compression)；
- **第三阶段 (07–08) 持久化**：任务系统 (Task System，将任务写入 `.tasks` 目录) 与后台任务 (Background Task)；
- **第四阶段 (09–12) 多代理团队**：代理团队 (Multi-Agent Team)、团队协议、自主代理，以及 Git 工作树隔离 (Git Worktree)。



图 5: 左：第二阶段（规划与知识）展开；右：第四阶段（多代理团队）展开

5

⁴ 视频画面时间区间：00:02:32–00:02:38。

⁵ 视频画面时间区间：左 00:02:38–00:02:50，右 00:03:10–00:03:26。

在这四个阶段中，特别值得关注的是后台任务与多代理团队。后台任务允许主代理在派生子代理执行长任务的同时，继续处理其他工作，子代理完成后异步汇报结果。而多代理团队则通过 Git Worktree 等隔离机制，确保多个代理在各自独立的目录中工作，互不干扰。

容易忽略的安全细节

多代理团队的并行能力虽然强大，但如果缺乏有效的隔离机制，多个代理同时修改同一份代码或同一个目录，就可能导致状态冲突甚至数据损坏。Claude Code 选择 **Git Worktree** 作为目录隔离方案，正是为了在享受并行效率的同时，守住安全与一致性的底线。

2.5 本章小结

本章系统梳理了 Claude Code Web 端架构的五大正交关注点，以及源码解析系列 12 个章节的功能全景。我们看到，一个完整的 Agent 系统不仅需要强大的 LLM 作为推理引擎，更需要 Harness 在外部提供工具、知识、感知、动作与权限的全方位支撑。安全机制（沙箱隔离与操作审批）是 Harness 的基石，而 Agent Loop、子代理、上下文压缩、后台任务、多代理团队等模块，则共同构成了从单代理到多代理协作的完整能力谱系。

3 核心机制：Agent Loop、工具管理与上下文权限

3.1 Agent Loop：几乎所有 Agent 都逃不过的模式

如果要从 Claude Code 的源码中提炼出一个最核心的运行模式，那无疑是 **Agent Loop**（代理循环）。主讲人在视频中用一句非常口语化但极其精准的话概括了这一点：

Claude Code 的核心，几乎每个 Agent 都是一个 `while True`，逃不过这个。

这句话看似简单，却道出了当前绝大多数 Agent 系统的底层范式。Agent 并非一次性地完成推理并输出答案，而是在一个持续运转的循环中不断感知环境、调用工具、接收反馈、调整策略，直至任务完成或达到终止条件。

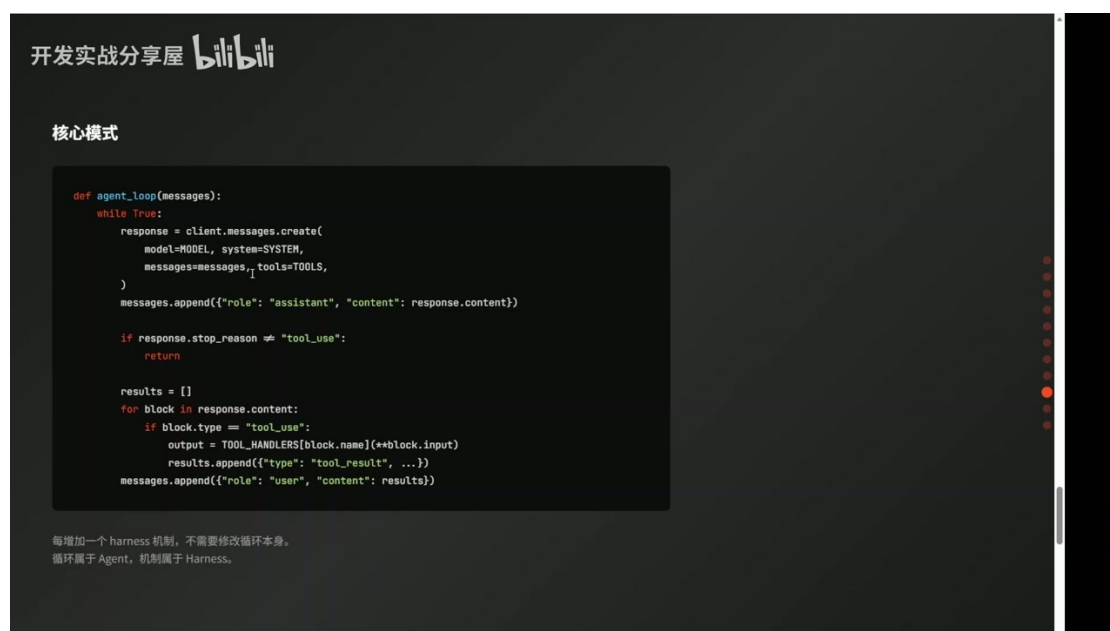


图 6: 核心模式代码示例: `agent_loop` 的 `while True` 循环结构

6

一个典型的 Agent Loop 可以抽象为以下伪代码:

```
1 while True:
2     response = llm.generate(context)
3     if response.has_tool_use():
4         result = tool_handler.execute(response.tool_use)
5         context.add_observation(result)
6     else:
7         break # 任务完成, 退出循环
```

Listing 1: Agent Loop 核心伪代码

在这个循环中, LLM 不断地根据当前上下文生成响应。如果响应中包含 `tool_use` block, Harness 就会调用对应的工具处理器 (Tool Handler), 将执行结果作为新的观察 (Observation) 追加到上下文中, 然后再次送入 LLM。如此往复, 直到模型不再需要调用工具为止。

3.2 Harness 的三大核心职责

既然 Agent Loop 是底层范式, 那么 Harness 在这个循环中扮演什么角色? 视频将 Harness Engineer 的核心职责归纳为三个方面: 工具管理、上下文权限、循环驱动。下图展示了更细化的四大职责视图。

⁶视频画面时间区间: 00:03:43-00:03:56。

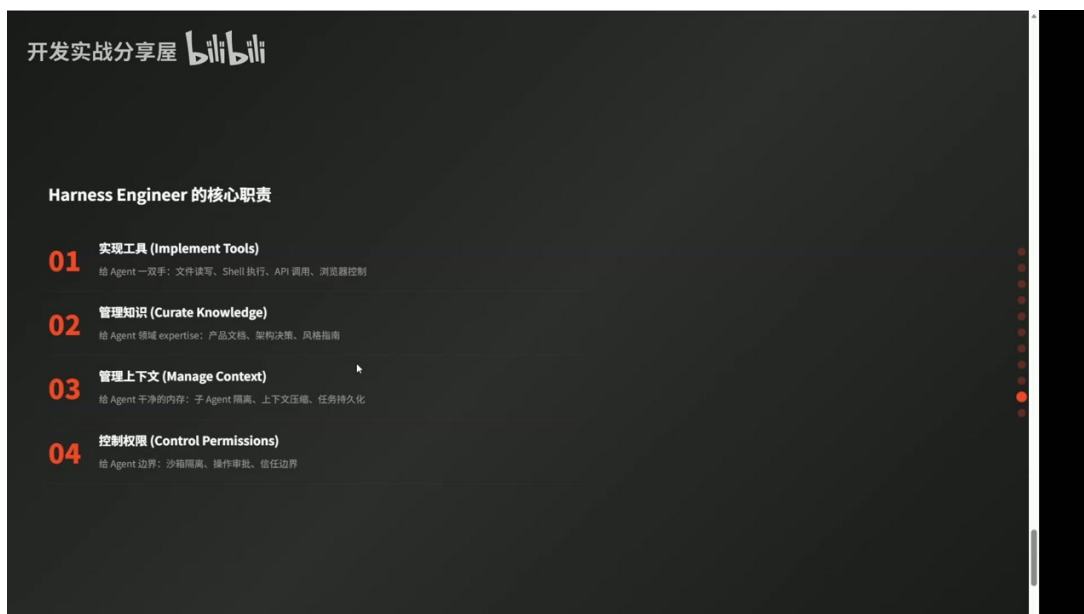


图 7: Harness Engineer 的四大核心职责：实现工具、管理知识、管理上下文、控制权限

7

Harness 的三大核心职责

1. **Agent Loop (循环驱动)**：持续驱动模型迭代，接收输入、生成响应、分发工具调用、回传结果；
2. **Tool Management (工具管理)**：决定模型能调用什么工具、如何注册工具、如何解析 `tool_use` block、如何处理执行异常；
3. **Context Permission (上下文权限)**：控制模型能看到什么上下文信息，以及它能执行哪些操作。

工具管理是 Harness 最显性的职责。没有工具，LLM 只是一台“会说话的机器”；有了工具，它才能读写文件、执行命令、查询数据库、调用外部 API。Harness 需要为每一个工具提供清晰的描述（description）、参数模式（schema）和执行逻辑（handler），以便 LLM 在生成 `tool_use` block 时能够准确选择并正确调用。

上下文权限管理则是 Harness 的隐性但同样关键的职责。上下文管理决定了 Agent 的“记忆”范围：是只读取当前文件，还是加载整个代码库？是保留全部对话历史，还是通过上下文压缩只保留关键信息？权限管理则决定了 Agent 的“行动边界”：哪些操作需要用户审批？哪些命令被沙箱禁止？哪些目录可以读写？

3.3 统一目标：让 LLM 始终往正确的方向走

Agent Loop、工具管理、上下文权限——这三大职责并非孤立存在，它们共同服务于一个统一的目标：**让大模型往正确的方向靠拢。**

⁷ 视频画面时间区间：00:03:56–00:04:05。

LLM 的偏离风险

如果没有 Harness 的约束，LLM 生成的 `tool_use` block 可能出现以下偏离：调用不存在的工具、传入错误的参数、在错误的目录下执行命令、或者重复执行已经成功的操作。Harness 的职责就是在这些偏离发生之前或之时进行拦截和纠正。

正如主讲人所总结的：

如果 LLM 本身的方向正确，那最好；如果不是，你就要通过外界的这些东​​西让它往正确的方向去走，返回的响应始终是你想要的。

这句话精准地概括了 Harness Engineering 的终极使命。Agent Loop 提供了持续纠偏的机会窗口，工具管理确保模型能够正确地与外界交互，上下文权限则限制了模型的视野和行动范围，防止它在信息不足或权限过大的情况下做出错误决策。三者协同，构成了一个完整的控制闭环。

3.4 本章小结

本章深入剖析了 Claude Code 的核心运行机制。我们认识到，`while True` 式的 Agent Loop 是当前 Agent 系统的基本运行范式；Harness 的三大核心职责——循环驱动、工具管理、上下文与权限控制——共同支撑起这一范式的可靠运转。这些机制的最终目标只有一个：无论 LLM 本身的输出是否完美，Harness 都能通过外部框架的引导、约束和纠正，确保 Agent 的行为始终朝着用户期望的方向前进。

4 学习展望：从 Claude Code 到更复杂的 Harness 工程

4.1 模型与 Harness 的解耦思想

在视频的后半段，主讲人将前面所有的讨论凝聚成了一个简洁而有力的公式：

$$\text{Agent} = \text{Harness} + \text{Model (LLM)} \quad (1)$$

这个等式虽然简单，却揭示了当前 Agent 系统设计中最重要​​的解耦思想。**Model (LLM)** 负责推理、规划与生成；**Harness** 负责控制、约束、工具调用与状态管理。两者各司其职，缺一不可。

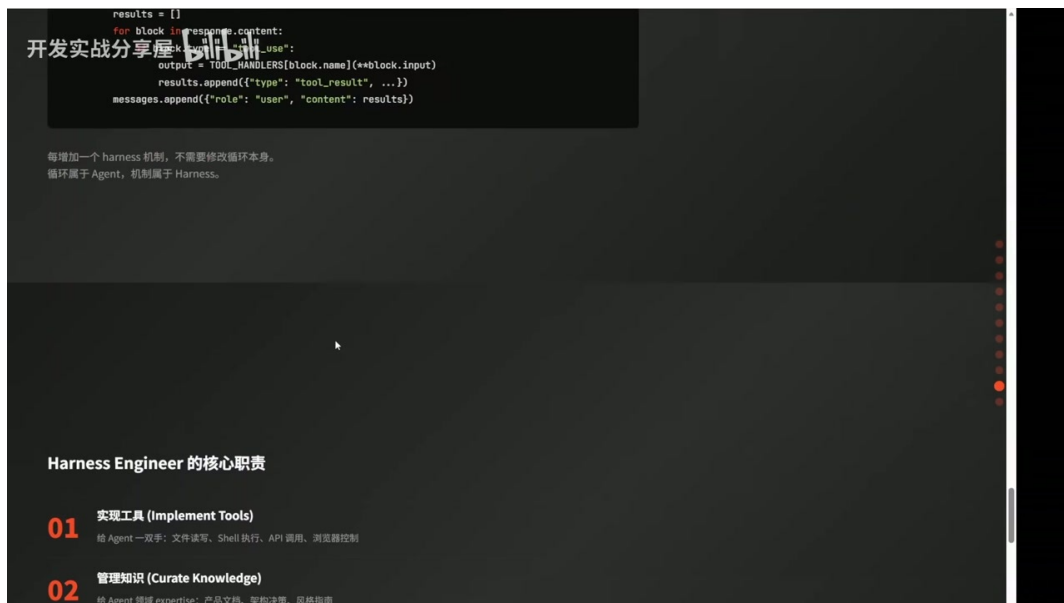


图 8: 总结页：模型即 Agent，代码即 Harness。构建优秀的 Harness，Agent 会完成剩下的工作。

8

视频中提到一种说法是”模型即 Agent，代码即 Harness”。更准确的理解应当是：Agent 是一个完整的系统，而 LLM 只是这个系统的”大脑”；真正的工程挑战在于如何为这颗大脑构建一个稳健、灵活、可扩展的 Harness。这正是 Claude Code 作为入门项目的最大价值所在。

4.2 Claude Code 的入门价值

对于希望深入理解 Agent 系统的开发者来说，Claude Code 是一个极佳的切入点。通过本系列的源码解析，我们已经系统地了解了 Harness Engineering 所涉及的核心组件：

- Agent Loop 的循环驱动机制；
- 工具调用（Tool Use）的注册、分发与异常处理；
- 权限控制（Permission）与沙箱隔离（Sandbox）的安全设计；
- 子代理（Subagent）与多代理团队（Multi-Agent Team）的协作模式；
- 上下文压缩（Context Compression）与任务持久化（Persistence）的优化策略；
- 后台任务（Background Task）与 Git 工作树隔离（Git Worktree）的并行架构。

这些知识点不仅适用于 Claude Code 本身，也是理解和评估其他 Agent 系统的通用语言。掌握了这些基础，我们就拥有了一张通往更复杂系统的路线图。

4.3 向更复杂系统进阶

视频在结尾部分展望了几条值得后续深入学习的方向：

⁸ 视频画面时间区间：00:04:16–00:04:28。

进阶学习路径

1. **OpenCode**: 开源的代码 Agent 项目, 适合作为 Claude Code 之后的第二个学习对象, 可以对比两者在 Harness 设计上的异同;
2. **OpenAI Codex**: 商业化的代码 Agent 系统, 功能更加丰富, 实现也更加复杂, 适合研究高级特性 (如更精细的权限模型、更复杂的规划算法);
3. **泄露版 Claude Code 源码**: 近期有部分 Claude Code 源码泄露, 其中包含了比本系列更底层的实现细节, 是理解 Anthropic 内部 Harness 工程实践的珍贵资料。

知识的可迁移性

无论是 OpenCode、Codex 还是泄露版 Claude Code, 它们在底层机制上与我们在本系列中学到的内容是相通的。Harness、Agent Loop、工具调用、权限控制等核心概念普遍存在, 区别主要在于实现的复杂度、工程规模和优化策略的不同。这意味着, 本系列所打下的基础, 可以直接迁移到后续的学习中。

4.4 本章小结

本章从“Agent = Harness + Model”这一核心等式出发, 总结了 Claude Code 作为入门项目的学习价值, 并展望了向更复杂 Agent 系统进阶的路径。我们认识到, Harness Engineering 是构建可靠 Agent 的基石, 而 Claude Code 恰好提供了一个结构清晰、模块完整的范本。接下来, 我们将通过 skills.sh 的实战演示, 看看这些 Harness 能力如何在具体场景中落地。

5 Skills.sh 实践演示与系列结语

5.1 Skills.sh: Agent 的能力市场

在视频的最后部分, 主讲人引入了一个极具实践价值的平台——**skills.sh**。这是一个面向 Agent 的 Skill 市场, 开发者可以在上面搜索、发现并安装各种预定义的能力包 (Skill)。当 Agent 需要完成某项特定任务时, 不再需要从零编写代码, 而是可以直接加载对应的 Skill, 让 Agent 自动调用其中的工具和工作流。

Skills.sh 的使用流程

1. 明确任务需求 (例如: 生成 PPT、设计网页、编写测试);
2. 到 skills.sh 搜索对应的 Skill (如 frontend-slides、frontend-design);
3. 将 Skill 安装到 VS Code 或本地环境;
4. Agent 自动识别并加载该 Skill, 完成任务。

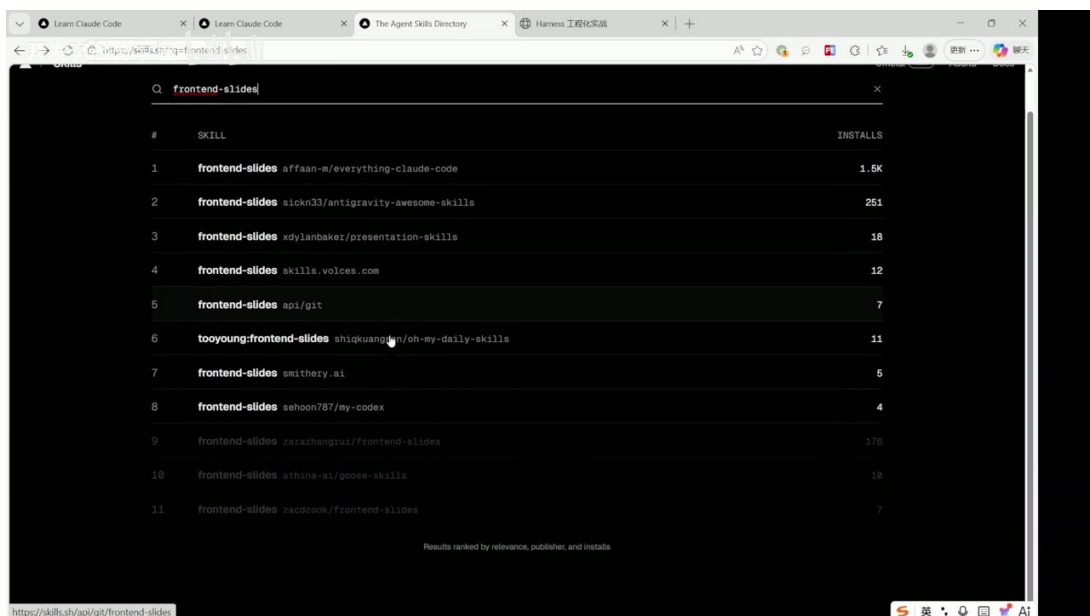


图 9: skills.sh 网站上搜索 frontend-slides 技能的结果列表

9

视频中使用的具体 Skill 名为 **Front End Slides**, 来自 GitHub 上的 zarazhangrui/frontend-slides。该 Skill 的风格非常简约, 能够根据给定的主题自动生成结构化的 slides/PPT。

5.2 Front End Slides Skill 实战演示

主讲人在视频中展示了 Roo Code (一款与 Claude Code 类似的 Agent 工具) 加载 frontend-slides Skill 的全过程。从搜索 Skill、查看详情页, 到安装并在 VS Code 中调用, 整个过程流畅自然。

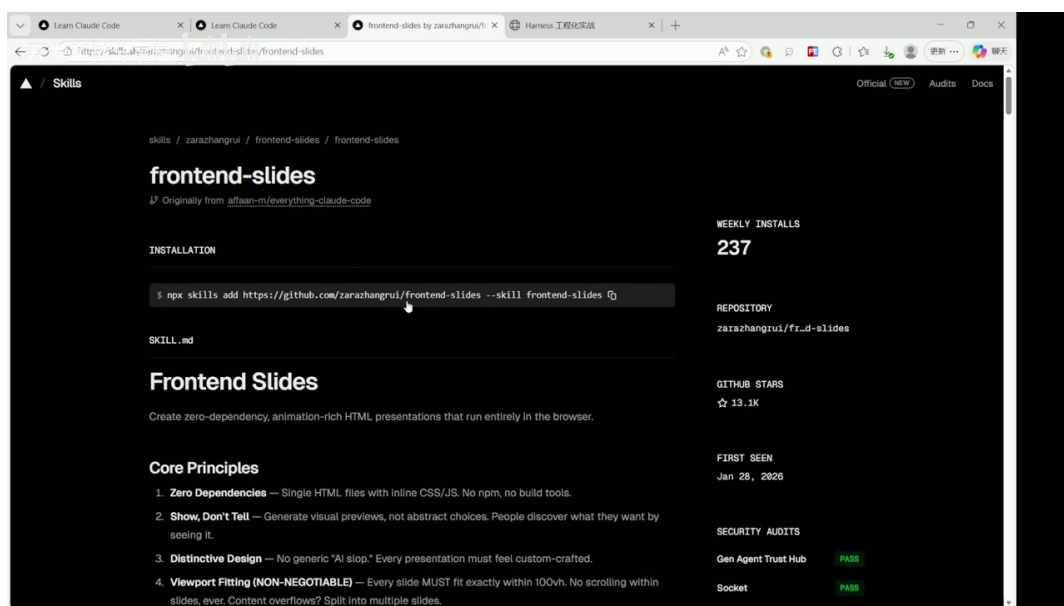


图 10: skills.sh 上 frontend-slides 技能的详情页, 包含安装命令与核心原则

10

⁹ 视频画面时间区间: 00:00:20-00:00:35。

更引人注目的是演示的后半段：Roo Code 在加载 Skill 后，自动读取了相关源码，并生成了一份完整的 PPT 大纲，涵盖了 Phase 1-4、Core Pattern、Harness Engineer 职责与 Summary 等模块。整个内容生成过程无需用户手动干预，充分体现了 Agent 根据 Skill 自动结构化内容的能力。

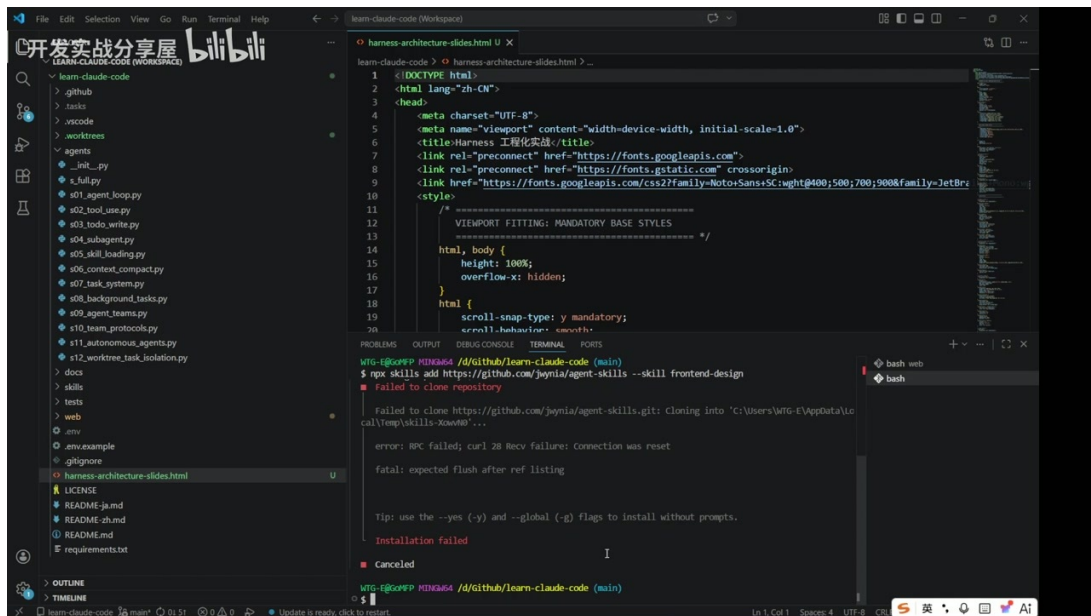


图 11: Roo Code 在 VSCode 中加载 frontend-slides 技能并自动生成 PPT 的实时演示

11

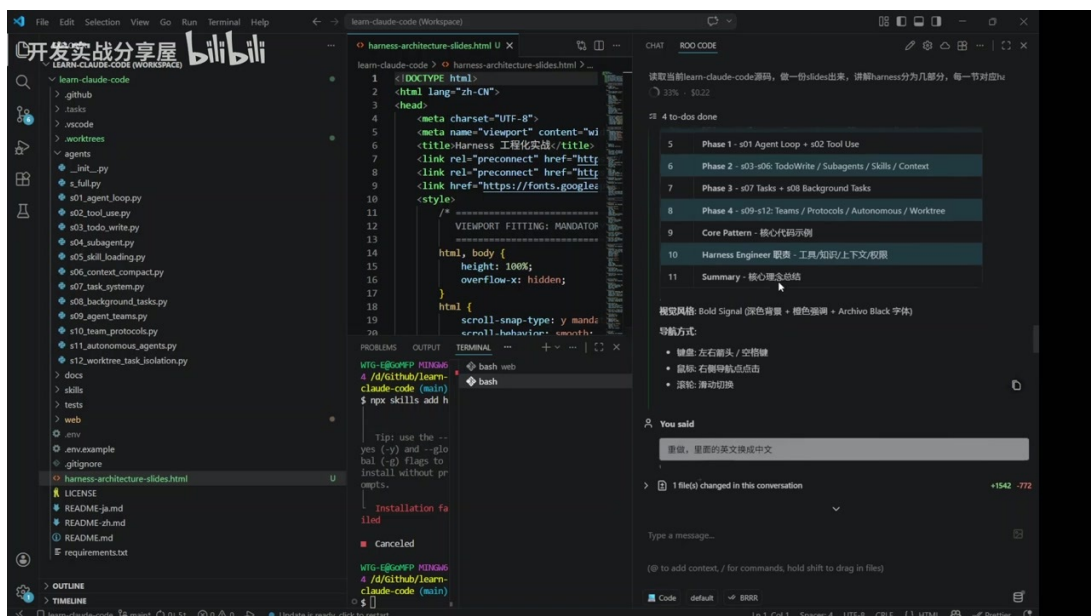


图 12: Roo Code 自动生成的 slides 大纲 (11 页)，展示 Agent 的结构化内容生成能力

12

¹⁰ 视频画面时间区间：00:00:26-00:00:35。

¹¹ 视频画面时间区间：00:05:58-00:06:05。

¹² 视频画面时间区间：00:06:05-00:06:15。

Agent + Skill 的内容生产趋势

无论是 Roo Code、Claude Code 还是其他 Agent 平台，它们都可以根据 Skill 创建出各种复杂或简约的 slides、网页、文档甚至代码库。这标志着 Agent 正在从“辅助编程”向“端到端内容生产”演进。对于开发者而言，学会发现和利用高质量的 Skill，将成为提升 Agent 生产力的关键技能。

5.3 系列结语

随着 Front End Slides 演示的结束，《Learn Claude Code 源码详解》系列也正式画上了句号。主讲人在最后表示，通过这个系列的学习，观众应当对 Claude Code 的 Harness Engineering 有了一个较为完整的认知，后续将会展开其他内容的讲解。

5.4 本章小结

本章介绍了 skills.sh 作为 Agent Skill 市场的定位，并通过 Front End Slides 的实战演示，展示了 Agent 如何借助 Skill 自动完成 PPT 生成任务。我们看到，Harness 的能力不仅可以内置在 Agent 系统中，还可以通过外部 Skill 进行无限扩展。这也意味着，未来的 Agent 开发将越来越像“搭积木”——选择合适的 LLM，构建稳健的 Harness，再叠加丰富的 Skill，即可组装出功能强大的智能应用。

6 总结与延伸

6.1 视频核心回顾

本视频作为《Learn Claude Code 源码详解》系列的完结篇，以 **Harness Engineering** 为核心线索，对 Claude Code 的 Web 端架构进行了一次高屋建瓴的总结。主讲人从“马具”这一生动隐喻出发，将 LLM 比作一匹能力强大但可能偏航的马，而 Harness 则是套住这匹马的缰绳与马鞍——它由工具、知识、感知、动作与权限五大组件构成，共同约束和引导模型的行为，使其输出始终符合用户的预期。

在此基础上，视频系统回顾了 Claude Code Web 端架构的五大正交关注点，以及源码解析系列 12 个章节的功能全景。从 Agent Loop 的核心循环，到子代理与多代理团队的协作；从上下文压缩与任务持久化，到沙箱隔离与操作审批——这些模块并非孤立的技巧，而是 Harness 这一整体框架下的有机组成部分。

6.2 从认知到实践的延伸思考

理解了 Harness Engineering 的框架后，我们可以从几个维度进行延伸思考：

1. **设计维度**：如何为不同的应用场景设计合适的 Harness？例如，面向代码生成的 Agent 需要强大的工具管理和沙箱隔离；面向对话陪伴的 Agent 可能更注重上下文记忆和知识注入；面向科学研究的多代理系统，则可能需要更复杂的任务调度与结果验证机制。
2. **安全维度**：随着 Agent 能力的增强，其潜在风险也在上升。视频中的操作审批和沙箱隔离只是起点，未来的 Harness 可能需要引入更细粒度的权限模型、行为审计日志、自动回滚机制，乃至对抗性测试框架。

3. **生态维度**: skills.sh 的出现预示了一个新兴的 Agent Skill 生态。未来的 Harness 工程可能不再是”从零造轮子”，而是”选择合适的 LLM + 搭建基础 Harness + 集成第三方 Skill”。这对于降低 Agent 开发门槛、加速应用创新具有重要意义。

6.3 下一步学习建议

如果你希望在本系列的基础上继续深入，以下是几条具体的学习建议：

推荐学习路径

- **精读 OpenCode 源码**：作为 Claude Code 的开源对标项目，OpenCode 在 Harness 设计上有很多值得对比研究的地方，尤其是它的工具注册机制和多代理调度策略。
- **关注泄露版 Claude Code 源码**：这些源码包含了比本系列更底层的实现细节（如 Prompt 工程、错误恢复逻辑、内部状态机），是理解 Anthropic 真实工程实践的珍贵资料。
- **实践 skills.sh**：挑选 2-3 个感兴趣的 Skill（如 frontend-slides、frontend-design），在 Claude Code 或 Roo Code 中实际安装并使用，体会”Skill 驱动开发”的工作流。
- **动手搭建最小 Harness**：尝试用 Python 实现一个最小的 Agent 原型，包含 while True 循环、简单的 Tool Handler、上下文截断和基础权限检查。实践是巩固理论的最佳方式。

Harness Engineering 不是一门可以速成的技艺，但它为我们理解、设计和评估 Agent 系统提供了一个坚实的概念锚点。愿你在从 Claude Code 到更广阔 Agent 世界的旅途中，始终记得这匹马与它的马具。